Chiron

Stabilization and Scope-Boundary Intensification After Runtime Migration

Progress Report 9 • 19 Jan 2026 — 30 Jan 2026

Master’s Research Progress Update

Fahad Alothman

Progress Report 9

Title: Stabilization and Scope-Boundary Intensification After Runtime Migration
Reporting Window: 19 Jan 2026 — 30 Jan 2026
Submission Context: Master’s Research Progress Update

Abstract

Progress Report 9 captures the period immediately after the main runtime migration wave. The central task of this window was not broad feature expansion; it was stabilization, behavioral verification, and architectural clarification under active execution pressure. The project focused on keeping the migrated runtime operational while resolving unresolved boundary questions that became visible during live workflow use.

A defining characteristic of this period was asymmetry between design intensity and repository change footprint in the exact reporting window. Runtime and UX behavior were exercised repeatedly through implementation sessions and seeded workflow runs, while core effort increasingly shifted toward contract clarification: step execution scope, approval semantics, tool-calling reliability, and model/provider variability handling.

---

1. Period Character: Post-Migration Stabilization Under Real Usage Pressure

This window should be interpreted as a stabilization-and-clarification phase after migration execution. The practical objective was to preserve continuity for independent workflows and workflow-path operation while addressing newly exposed reliability and authority-boundary issues.

The period therefore prioritized:

- runtime parity checks under real interaction flows,
- reliability corrections in streaming and approval behavior,
- and architecture decisions that reduced ambiguity at project, workflow, and step levels.

---

2. Operational Reality in This Window: Independent Workflows Still Driving Execution

Chiron was still running on independent workflows connected through workflow paths as the active execution backbone. This continuity mattered: migration work had altered runtime internals, but user-facing orchestration still depended on path-driven progression.

As a result, stabilization work concentrated on protecting this backbone from regression while hardening the boundaries required for later formal methodology-layer contracts.

---

3. Streaming and Tool-Calling Reliability as the Immediate Failure Surface

A major focus in this period was sandboxed-agent streaming behavior and tool-calling reliability across provider/model combinations [1], [2], [5]. In practical terms, this emerged as the most immediate reliability surface because it touched user-visible responsiveness, approval flow timing, and tool-argument fidelity.

The project's response in this window emphasized robust operational behavior over idealized assumptions:

- validate streaming/event ordering and completion semantics,
- strengthen fallback behavior when provider/tool-schema behavior diverges,
- and align runtime handling with real model variance instead of static capability assumptions.

This moved the system toward resilience-oriented orchestration rather than provider-specific optimism.

---

4. Step Execution Scope Becomes a First-Class Architecture Topic

A central architectural development in this period was explicit intensification of step-level scoping discussions. The project increasingly treated step execution as the practical unit of interaction, approval, and context mutation, distinct from broader workflow execution state.

This shift sharpened multiple decisions:

- what belongs to workflow execution versus step execution,
- how approval states should be represented consistently across step types,

- how tool call records, chat records, and step-level coordination data should be separated without losing traceability.

These concerns were not theoretical. They emerged from active runtime troubleshooting and therefore carried direct implementation consequences.

---

5. Boundary Design Pattern in This Window: Operational Contracts Before Full Formalization

As in the previous report, formal methodology-layer contracts were not yet the primary execution abstraction. During this period, the project continued to use operational contracts to enforce control:

- scoped execution context handling,
- explicit transition and approval authority boundaries,
- service-level separation for runtime concerns (execution, chat, approvals, tooling/event flow).

What changed in this report window is intensity. Boundary questions moved from “emerging concern” to “active architecture workstream,” with implications for later schema and module-shape decisions.

---

6. Methodology Preparation During Stabilization

Although this period did not complete the final methodology-layer materialization, it improved readiness for methodology-layer alignment in two practical ways:

1. by testing migrated runtime behavior against real seeded workflow paths and execution scenarios,
2. by translating observed runtime failure modes into clearer boundary and contract requirements for subsequent architecture iterations.

In this sense, stabilization served as methodology preparation: it converted operational friction into explicit design constraints.

---

7. Stack Status at End of Progress Report 9

At the end of this reporting window, the runtime direction remained aligned with the migration target established in Progress Report 8:

- **Desktop host:** Tauri [8]
- **Frontend:** React + TanStack Router + shadcn/ui
- **Backend/API:** TypeScript services with Hono + tRPC [6], [7]
- **AI/runtime orchestration path:** Effect + AI-SDK (+ AX/provider integrations under active refinement) [1]–[5]
- **Data layer:** Drizzle ORM with PostgreSQL
- **Authentication:** Better Auth
- **Runtime/tooling:** Bun + Turborepo

The key historical signal in this period is not a new stack pivot; it is deeper boundary refinement and reliability hardening inside the new runtime direction.

---

8. Why This Period Matters

Progress Report 9 matters because it transformed migration-aftershock issues into architectural signal. Instead of masking instability, the period surfaced where authority, lifecycle, and scope boundaries were still under-defined, then used those observations to shape the next design layer.

This preserved continuity for current workflows while improving readiness for the later formal methodology-layer and lifecycle/state contract work.

---

9. Open Questions Entering Progress Report 10

- Which step-execution boundaries should be formalized as durable schema and contract primitives?
- How should approval semantics be unified across step types without coupling UI flow to runtime internals?
- What provider/model variance policy should be standardized so streaming and tool-calling remain reliable across runtime contexts?
- Which operational contracts are now stable enough to promote into explicit methodology-layer constructs?

These questions define the handoff from stabilization-driven clarification to explicit contract formalization.

Conclusion

Progress Report 9 is best understood as a boundary-intensification period after migration execution. The project remained operational on independent workflows and workflow paths while confronting the practical limits of newly migrated runtime behavior in streaming, approvals, and step-level authority.

Its core contribution is architectural sharpening: it converted post-migration reliability friction into clearer scope and lifecycle design requirements, setting up the next phase of formal contract and methodology-layer maturation.

References

[1] Effect Authors, “Effect Documentation,” 2026. [Online]. Available: https://effect.website/
[2] Vercel, “AI SDK Documentation,” 2026. [Online]. Available: https://ai-sdk.dev/docs
[3] Vercel, “AI SDK 6,” 2025. [Online]. Available: https://vercel.com/blog/ai-sdk-6
[4] axllm, “ax,” GitHub repository (`@ax-llm/ax`), 2026. [Online]. Available: https://github.com/axllm/ax
[5] OpenCode, “OpenCode,” GitHub repository, 2026. [Online]. Available: https://github.com/anomalyco/opencode
[6] Hono Authors, “Hono,” Documentation, 2026. [Online]. Available: https://hono.dev/
[7] tRPC Authors, “tRPC,” Documentation, 2026. [Online]. Available: https://trpc.io/docs
[8] Tauri Programme, “Tauri,” Documentation, 2026. [Online]. Available: https://tauri.app/